

LAB Assignment 4

Statement Purpose:

The objective of this lab is to familiarize the students with graphical user interface development. For this purpose, **JavaFX** framework will be introduced to students in this Lab. various concepts relevant to **JavaFX** application and its skeleton will be introduced to students. The students will also get hands on experience on drag-n-drop based software namely **Scene Builder**, to build **JavaFX** scene based on **FXML** language.

Activity Outcomes: (CLO4-PL05/P4)

After completion of this lab, students will be able to

- understand how Java applications are different from **JavaFX** applications.
- understand two basic structures of implementing **JavaFX** applications.
- learn how to implement classes related to **JavaFX** package and its sub-packages.
- design a GUI in **Scene Builder**
- Understand MVC design pattern for **JavaFX** application with **FXML**.
- Implement event-based programming o respond to user interactions.

Software Resources

- Jave SE 8 or higher, IDE IntelliJ Idea
- **JavaFX** libraries

Note: Systems are provided for students in the 1:1 ratio

Instructor Note:

The students should have prior knowledge and practiced

- Inheritance, polymorphism, and interfaces
- Basic understanding of exceptions and generics may also be required to understand the code.

Suggested Prelab Reading

Java's UI frameworks include **Java AWT**, **Java Swing**, and **JavaFX**. These are all toolkits used to build Graphical User Interfaces (GUIs) for Java applications. System dependent AWT (Abstract Window Toolkit) is very primitive original toolkit, system independent Swing is its more powerful extension, and the latest toolkit JavaFX is the modern replacement of both the earlier toolkits.

JavaFX is the latest Java-based framework offered by Oracle. **JavaFX** provides a powerful and flexible framework that simplifies the creation of modern GUIs based applications. The original **JavaFX** was based on a scripting language called **JavaFX Script**. However, it has been discontinued. Since **JavaFX 2.0**, **JavaFX** has been programmed in Java itself and provides a comprehensive API to develop GUI based applications. **JavaFX** also supports **FXML** language to build the view of the application. **JavaFX** also supports model-view-controller architecture (MVC).

To use AWT components, import `java.awt` package. To use Swing components, you have to import `javax.swing` package and in order to use JavaFX components, you import `javafx`.

Note: You also require some `import` instructions at the start of the code (refer to lecture slides for these imports). Be careful of the `import` statements while typing the code as some `javafx` classes identifiers are same as in `java.awt` API, always select the correct version by selecting `javafx` version). If you observe some import statements of type `java.awt` or `javax.swing`, retype the component type and select the one that come with `javafx` from the list proposed by IDE.

a. Pre-requisite

JavaFX builds on top of JDK and is a standalone component. There are 2 different options for developing **JavaFX** applications:

- Use the **JavaFX SDK** (choosing between 17 or 21 LTS, latest release 22, or an early access build).
- Use a build system (e.g. **Maven/Gradle**) to download the required modules from **Maven Central** (choosing as well between the same mentioned versions).

In any case, for both of the above options, it is required to have a recent version of **JDK 21**, or at least **JDK 17**.

Download JavaFX SDK

If you want to use the **JavaFX SDK** instead of a build tool, download an appropriate **JavaFX** runtime for your operating system from the following link and unzip it to a desired location.

<https://gluonhq.com/products/javafx/>

- i. Download **JavaFX SDK** (found in compressed format)

For windows, downloaded file name for JDK21 would be **openjfx-21.0.1_windows-x64_bin-sdk.zip**.

- ii. Unzip **JavaFX SDK** folder

After unzip, the uncompressed folder name would be **javafx-sdk-21.0.1**

- iii. Move the **JavaFX SDK** folder to **.../java/** folder containing **Java JDK** (It is optional step)

Move **javafx-sdk-21.0.1** to **C:\Program Files\Java**

- iv. With that, now the **JavaFX** library path will be **C:\Program Files\Java\javafx-sdk-21.0.1\lib** (which will be used later)

Creating and Configuring JavaFX Project in IntelliJ IDEA

Proceed through following steps to create a java project:

- select **New Project**
- Enter Name, for example → **labTasks**
- Enter Location, for example → **d:\myWorkspace**
- Select Language → **Java**
- Select Build System → **IntelliJ**
- Check → Add Sample Code
- Click Create → **Create**

In the next step, extend our **Java** application to an **JavaFX** application by proceeding through the following procedures:

1. Add JavaFX Libraries (Step 1)

- Go to **Project Structure** in **File** menu
- Further go to **Global Libraries** under **Platform Settings**
- Click on + sign on the right viewlet → +, Select **Java** → **Java**
- Under **Select Libraries Files**, Give path to **JavaFX** libraries on your system, i.e., "**C:\Program Files\Java\javafx-sdk-21.0.1\lib**"
- Click **OK**
- Click **OK**

2. Add JavaFX Libraries (Step 2)

- Go to **Project Structure** in **File** menu
- Further go to **Modules** under **Project Settings**
- Click on + sign on the right viewlet under **Dependencies** tab → +
- Select **Library...**, Chose **lib**, under **Global Libraries** in **Choose Libraries** window
- Click **Add Selected**, Click **OK** if ask to confirm the action.

Developing JavaFX Applications

There are two approaches to develop JavaFX applications.

1. **JavaFX Application Without FXML Based User Interfaces**
2. **JavaFX Application With FXML Based User Interfaces**

Lab Tasks:

Task 1: JavaFX Application without FXML based User Interfaces

Create a java package hierarchy `labs.lab12.task1`. Create a Java class **Main** in this package and extend it from **Application** class to build **JavaFX** application.

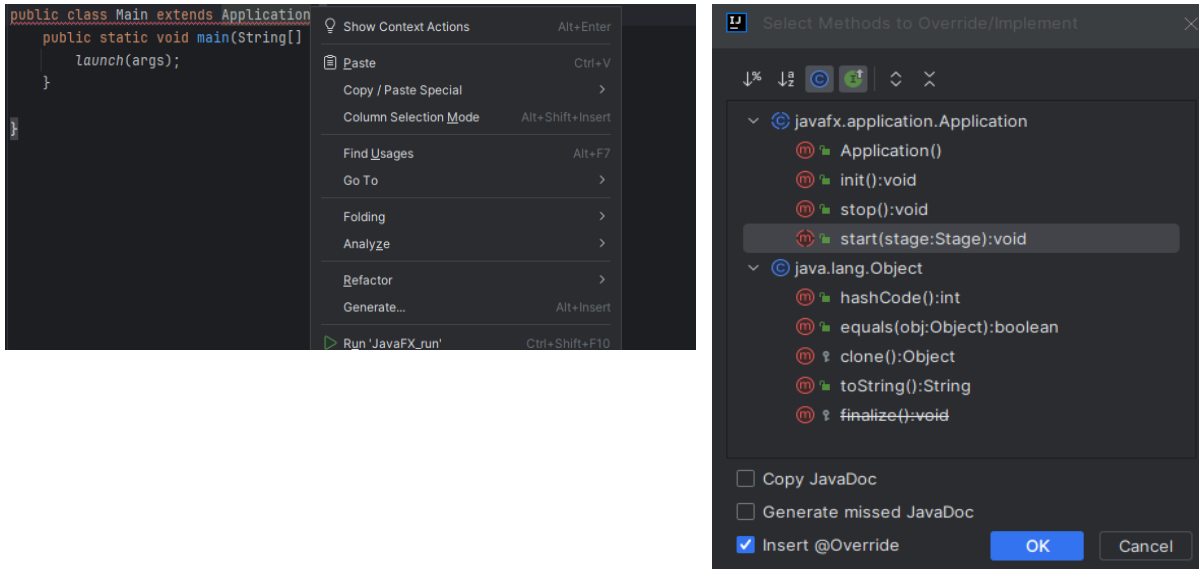
Extending **Main** class from **Application** class

- Extend your main class (class having **main** method) from class **Application** (as described in the slides)
- You will find swingy lines (___) below class header. As the main class extends **Application** class, the main class must implement an **abstract** method **start()** of the **Application** class.

Once you have extended the main class from the **Application** class, perform the following steps.

- Add following instance variables to your main class outside any method.
`Label response;`
`ToggleGroup tg;`
- Add a call to the method **launch** in the **main** method of the class as shown below:

```
public static void main (String[] args) {  
    launch(args);  
}
```
- Create the **start** method by using **Generate...** from the context menu appeared when right clicked after the **main** method and **Generate...** > **Override Methods...** then selecting **start (stage: Stage):void** under **javafx.application.Application**.



- A skeleton for **Override start()** method will appear after the **main** method.

```
@Override
public void start(Stage stage) throws Exception {

}
```

- In **start()** method, configure the **stage** and create an instance of **FlowPane** as given in below:

```
stage.setTitle("Lab JavaFX");
FlowPane rootNode = new FlowPane (10, 10); //vertical and horizontal gaps of 10
rootNode.setAlignment (Pos.CENTER);
```

- In **start()** method, also create a **Scene** instance from **FlowPane** instance as shown below:

```
Scene myScene = new Scene (rootNode, 300, 180);
stage.setScene (myScene);
```

- In **start()** method, also create desired labels, buttons, horizontal separator and radio buttons as shown below (the radio buttons which can be exclusively selected must be part of the same toggle group – i.e. instance of class **ToggleGroup**):

```
Label choose = new Label (" Select your semester");
response = new Label ("No semester mentioned.");
Button btnConfirm = new Button ("Confirm Semester Selection");
RadioButton rbOne = new RadioButton ("One");
RadioButton rbTwo = new RadioButton ("Two");
```

```

RadioButton rbThree = new RadioButton ("Three");
tg = new ToggleGroup ();
rbOne.setToggleGroup (tg);
rbTwo.setToggleGroup (tg);
rbThree.setToggleGroup (tg);
rbOne.setSelected (true);
Separator separator = new Separator ();
separator.setPrefWidth (190);

```

- In **start()** method, add all GUI components (nodes) to the root node and set the stage to display:

```

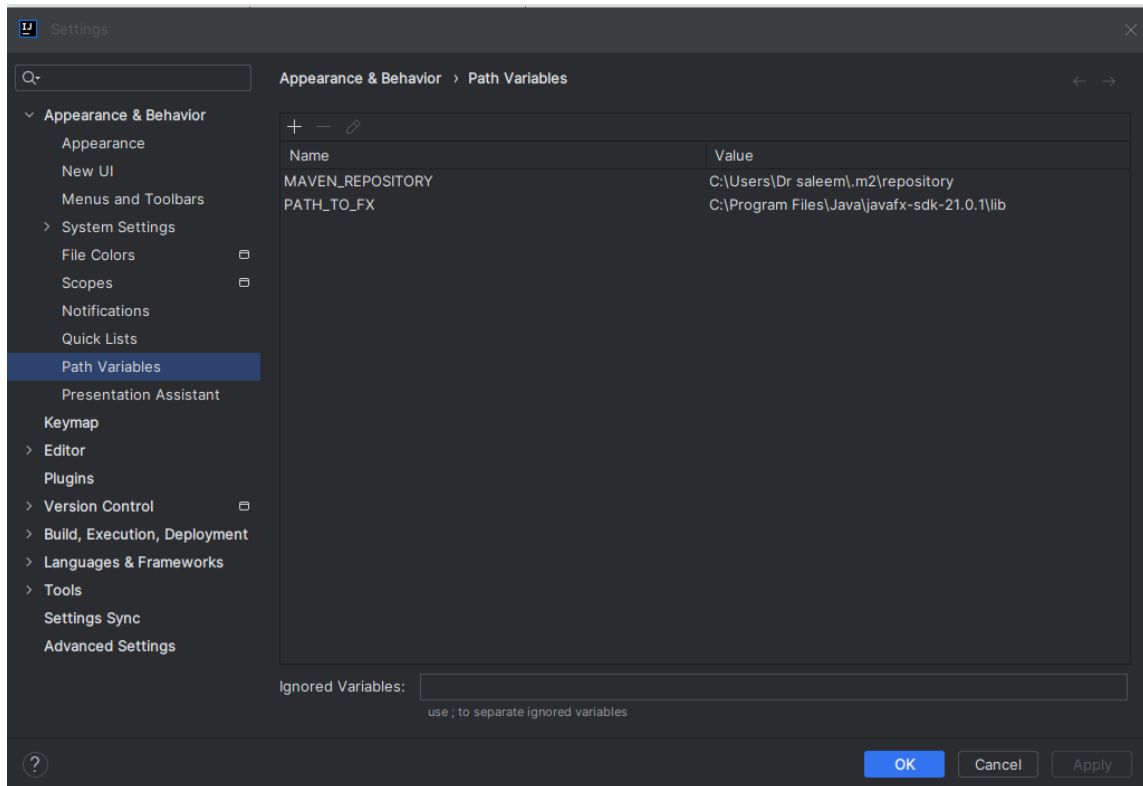
rootNode.getChildren().addAll (choose, rbOne, rbTwo, rbThree,
    separator, btnConfirm, response);
stage.show();

```

Build and Run JavaFX application

1. Adding **PATH Variables**.

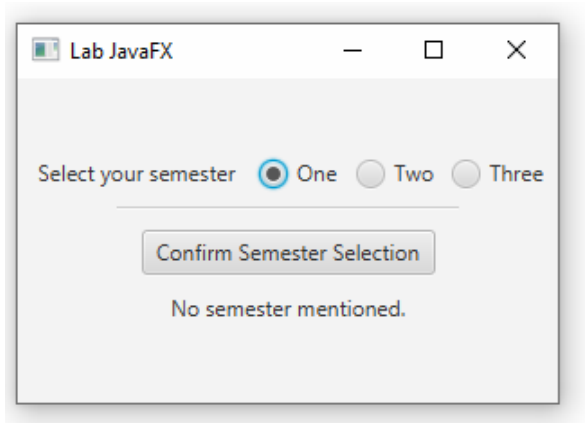
- Go to **Files** menu → **Settings** → **Appearance & Behavior** → **Path Variables**, Click + sign and enter the name of variable as **PATH_TO_FX** and Its value set to **JavaFX** libraries folder. Here, **C:\Program Files\Java\javafx-sdk-21.0.1\lib**. Click **OK** to confirm and save.



2. Add or Edit **Run Configurations**

- Go to **Run** menu → **Edit Configurations...**
- Click **Add** new run configuration → **Application**
- Set appropriate **Name** → **JAVAFX_Build&Run_1**
- Click **Modify** Options and then **Add VM Options**
- Set **VM Options** → `--module-path "$PATH_TO_FX$" --add-modules javafx.controls,javafx.fxml`
- Set **Main Class** (must be fully-qualified path) → **Main**
- Click green **PLAY** button to execute **Run Configuration**, namely **JAVAFX_Build&Run_1**

The Expected Output



Adding Event Handlers

When you click the button “**Confirm Semester Selection**” after selecting semester “**One**”, “**Two**” or “**Three**”, it does nothing. Therefore, register the following event listener before adding all nodes to the root node.

```
btnConfirm.setOnAction (new EventHandler<>() {  
    public void handle (ActionEvent ae) {  
        RadioButton rb = (RadioButton) tg.getSelectedToggle ();  
        response.setText (rb.getText() + " is confirmed.");  
    }  
});
```

Run the application again. After selecting semester “**One**”, “**Two**” or “**Three**”, click the button “**Confirm Semester Selection**”, to see the response.

Task 2: JavaFX Application with FXML based User Interfaces created

Another approach to develop **JavaFX** is to use **FXML** language to convey information about UI components. Again, writing the complete GUI in **FXML** would be a tedious job. To ease **GUI** developers, the drag and drop application – **Scene Builder** – is available to develop the **GUI** with ease and in less time.

When you use **FXML** file in your application, you need at least the following three files to make an interactive application.

- A main class (class with **main** method) file which represents your application i.e. **ClassName.java** where **ClassName** extends **Application** class.
- A layout file e.g. **MyLayout.fxml** which describes the scene graph.
- In case of an interactive application, you need another file caller controller file of the scene graph e.g. **MyLayoutController.java** which has access to the controls and other containers in the UI defined in layout **fxml** file describing layout details (i.e. **MyLayout.fxml**).

Creating main java File

Create a java package hierarchy **labs.lab12.task2**. Create a Java class **Main** in this package and extend it from **Application** class to build this **JavaFX** application. Follow the instructions described in **Task 1**. Once you have extended your main class from the **Application** class, perform the following steps.

- Add a call to launch method in the **main** as shown below:

```
public static void main(String[] args) {
    launch(args);
}
```

- Create an **Override** method **start** following the instructions described in Task 1.

```
@Override
public void start (Stage stage) throws Exception {

}
```

- In **start** method, configure the **stage** and create an instance of **FXMLLoader** class as given in below (be careful of the **import** statements while typing the code as some **javafx** classes identifiers are same as in **java.awt API**, always select the correct version by selecting **javafx** version)

```
stage.setTitle("Lab JavaFX from FXML");

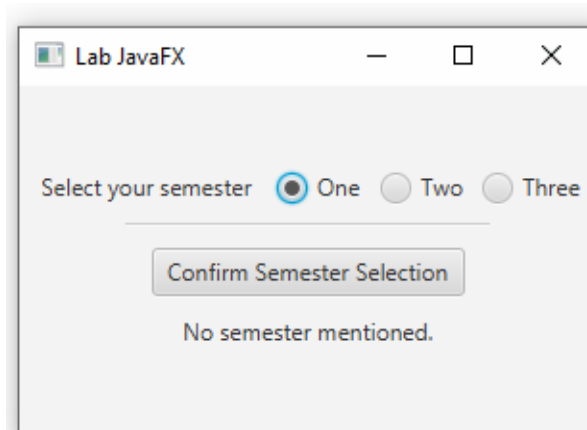
FXMLLoader loader = new
FXMLLoader(getClass().getResource("MyLayout.fxml"));
Parent rootNode = loader.load();
Scene myScene = new Scene(rootNode);
stage.setScene(myScene); // Set scene on stage
stage.show(); // Display window
```

Creating fxml File from Scene Builder

Scene Builder tool, enables you to create graphical user interfaces (GUIs) with drag and drop techniques. You can download Scene Builder from:

<https://gluonhq.com/products/scene-builder/>

Open **Scene Builder** and drag and drop components to make a design like the following GUI diagram:



To create above scene, proceed through following steps to insert scene nodes:

- Insert **Flow Pane** container from **Containers** menu list into the content panel of **Scene Builder**. Just double click the **Flow Pan**, it will automatically insert a **Flow Pan** box over “**Drag Library items here...**” box.
- Insert **Label** control from **Controls** menu list into the **Flow Pan** box by double clicking **Label**. It will automatically insert a **Label** component in **Flow Pan** box near its top left corner. Select the inserted **Label** component and set its **Text** property to “**Select your semester**” by selecting a sequential dropdown menu (**Inspector > Properties > Text**). Observe the change in component label in **Flow Pan**.
- Insert three **Radio Button** controls from **Controls** menu list into the **Flow Pan** box one by one by double clicking **Radio Button**. **Radio Button** components will be placed automatically within **Flow Pane next to each other and after the label “Select your semester”**. Select the **Radio Button** components one by one and set their **Text** property to “**One**”, “**Two**” and “**Three**”, respectively by following the similar steps as that for the **Label** control “**Select your semester**”.

- Double click **Separator (Horizontal)** control from **Controls** menu list to insert it into the **Flow Pan** box.
- Double click **Button** control from **Controls** menu list to insert it into the **Flow Pan**. Button component will be placed automatically within **FlowPane**. Select **Button** component and set its **Text** property to “**Confirm Semester Selection**” by following the similar steps as that for the **Label** control “**Select your semester**”.
- Similarly, insert another **Label** control from **Controls** menu list into the **Flow Pan**. Select the **Label** component and set its **Text** property to “**No semester mentioned**”.

Configure other properties of the scene nodes for better aesthetics as described below:

- Select the **Flow Pane** component (outer box) and configure the following properties by selecting sequential dropdown menus **Inspector > Layout** and **Inspector > Properties** respectively.

Inspector – Layout	Value	Inspector – Properties	Value
Pref Width	300	Alignment	CENTER
Pref Height	180		
Hgap	10		
Vgap	10		

- Select the **Separator** component and configure its following **Layout** property by selecting sequential dropdown menu **Inspector > Layout**. **Separator** component will be readjusted automatically within **FlowPane**.

Inspector – Layout	Value
Pref Width	190

In the menu bar of the **Scene Builder**, click **Preview** → **Show Preview** in Window

This will show, how your scene will look like in real window when executed through **JavaFX** application.

Creating Controller File from Scene Builder

Proceed through following steps to create the skeleton of the **Controller** class for the **fxml** file (i.e. **MyLayout.fxml**) so that nodes within scene created through the **fxml** can be modified when desired.

- Set **Controller** class name as “**MyLayoutController**” in **Scene Builder** by adding name of the **Controller** class by selecting sequential dropdown menus **Document > Controller** on the bottom left

side of the **Content** panel.

Note: As you are working for this lab task in the package, `labs.lab12.task2`, you would need to add the fully-qualified path for the **Controller** class name as "`labs.lab12.task2.MyLayoutController`".

- In the menu bar of the **Scene Builder**, click **View** → **Show Sample Controller Skeleton**. This will show **Controller** class for the **JavaFX** application.
- Add identifiers to all those scene nodes whom you want to modify when application is running (e.g. on certain actions or events). The identifiers (for objects) can be added in the **fx:id** attribute in the **Inspector – Code** panel of a certain scene node. Select following scene nodes one by one and set **fx:id** values given below by selecting sequential dropdown menu **Inspector > Code**:

Scene Node	fx:id
Label with text "No semester mentioned."	response
Button with text "Confirm Semester Selection"	btnConfirm

By now, the **Controller** class should look as follows when you click **View** → **Show Sample Controller Skeleton**:

```
package labs.lab12.task2;
import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
public class MyLayoutController {
    @FXML
    private Button btnConfirm;
    @FXML
    private Label response;
}
```

- Until now, you can select multiple **Radio Buttons** simultaneously. Assign same toggle group to each radio button to select only one out of three at a time. Select each **Radio Button** one by one and set following property on it by selecting sequential dropdown menu **Inspector > Properties**:

Inspector – Properties	Value
------------------------	-------

Toggle Group	tg
--------------	----

Also check (tick/ mark) **Radio Button “One”** as **Selected** in **Inspector – Properties – Specific**.

By now, the **Controller** class should look as follows when you click View → **Show Sample Controller**

Skeleton:

```
package labs.lab12.task2;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.ToggleGroup;

public class MyLayoutController {

    @FXML
    private Button btnConfirm;

    @FXML
    private Label response;

    @FXML
    private ToggleGroup tg;

}
```

- Until now, the **Button** with text “**Confirm Semester Selection**” is doing nothing when you click it. Select the button and add the following event handler method to **on Action** event (i.e. click event) by setting the value in **Inspector – Properties – Main** property.

Inspector – Code – Main		Value
On Action		onClickHandler

- Click the **View → Show Sample Controller Skeleton** to see updated controller class. It should now include method

```
void onClickHandler(ActionEvent event) { }
```

- On the sample skeleton window of **Controller** class, click on the button “**Save as...**”. Select path to save the file in the same package of your current task (i.e., **labs.lab12.task2**). Name the file as “**MyLayoutController.java**” and save it.
- On **Scene Bulder**, select **File** menu, click **Save As . . .**. Save the generated **fxml** file with by naming it

as `"MyLayout.fxml"` in the same package of your current task (i.e., `labs.lab12.task2`).

Updating Controller Class in IntelliJ IDEA

In this step, handler functions in the **Controller** class `MyLaoutController` will be defined so that they can perform some operations.

Update `onClickHandler` method as follows:

```
@FXML
void onClickHandler(ActionEvent event) {
    RadioButton rb = (RadioButton) tg.getSelectedToggle();
    response.setText(rb.getText() + " is confirmed.");
}
```

Build and Run JavaFX application

1. Add or Edit **Run Configurations**

- Go to **Run** menu → **Edit Configurations...**
- Click **Add** new run configuration → **Application**
- Set appropriate **Name** → `JAVAFX_Build&Run_2`
- Click **Modify** Options and then **Add VM Options**
- Set **VM Options** → `--module-path "$PATH_TO_FX$" --add-modules javafx.controls,javafx.fxml`
- Set **Main Class** (must be fully-qualified path) → `Main`
- Click green **PLAY** button to execute **Run Configuration**, namely `JAVAFX_Build&Run_2`

Run the application. Its behavior must be the same as of Task 1.

Configure JavaFX Scene Builder for IntelliJ IDEA

IntelliJ IDEA allows you to open `.fxml` files in **JavaFX Scene Builder** right from the IDE after you specify the path to the **Scene Builder** application in **IntelliJ IDEA** settings. For integration of **Scene Builder** with

IntelliJ IDEA, follow the instructions on the following link.

<https://www.jetbrains.com/help/idea/opening-fxml-files-in-javafx-scene-builder.html>